EVO

The boundary layer for AI agents

A build plan for one day, from zero, with the clock running.

EVENT	Ship Happens · AWS Student Builder Group × ADIGPS
DATE	Saturday 31 October 2026 · 9:00 – 18:30 IST
VENUE	Juspay Technologies · Gurugram
FREEZE	4:00 PM · demos 4:30 · awards 6:00
TRACK	Kiro (AWS) — consumes Evo over MCP

Everyone in that room will build an agent that does something.
You are building the thing the agents have to ask.

That is the only category of one available today.

Revision 2 — Sarvam credits did not land. The track integration is Kiro consuming the boundary over MCP; Block D, the demo and the submission below reflect that.

PART ONE

What you are actually entering

A hackathon is a compression contest: how much credible uniqueness can you put in front of a tired judge in three minutes. You already have more substance than most teams will produce today — a deterministic engine, a real corpus, a verifier that fails the build when it is wrong.

What you are not doing today is writing more engine. You are giving it a mouth.

- A service. Something on a port with endpoints. Not a page.
- A protocol. Any agent can ask — and one does, on stage, in an AWS IDE.
- A UI. What a human sees when they ask the same question.

Three things a judge decides in thirty seconds

	The question in their head	What answers it
1	Do I understand this?	One sentence, repeated three times, never varied.
2	Is it real?	Real events, real numbers, a failure you cause on purpose.
3	Is it theirs?	A refusal, a gate, an agent asking. Things you cannot fake live.

> **THE UNFAIR ADVANTAGE**
> Every other team will demo an agent that does a task. When they are asked "how does it know what to work on?", they will say "the user tells it" or "we pass the folder". You are the answer to that question. Do not compete on what the agent does. Compete on what it is allowed to touch.

What wins, specifically

Judges reward four things, in this order: a demo that does not break, a problem they recognise, evidence that it is not a wrapper, and a story they can repeat. Novelty is third. You do not need to be the most original project in the room — you need to be the one nobody can place in an existing category.

Judging pressure	How Evo answers it
"Is this another OpenAI wrapper?"	The decision path has no model in it. Turn off the wifi and it still works.
"Isn't this just a folder?"	A folder is declared in advance. This is derived from behaviour, and carries its own evidence.
"How do I know it's right?"	It refuses six of eight works rather than guessing, and shows what it rejected.
"Why now?"	Agents just started acting on real machines. They need jurisdiction first.

The most memorable thing you will do today: let a judge try to merge two works, refuse them, then let them declare it anyway and watch the build fail. People remember the product that told them no.

PART TWO

The demo product and the real product

These are the same thing at two distances. Say it that way and the ambition lands without sounding like a deck.

	Demo product (today)	Real product (what you say next)
What it is	A local service on one laptop. Any program on the machine can ask which body of work a resource belongs to.	The same service shipped inside the desktop app. Every agent on the machine queries it as infrastructure.

Who uses it	You, in front of a judge, in three minutes.	Knowledge workers whose agents act on their machines.
How it earns	Nothing.	Per seat, prosumer first, then team. The boundary is the moat, not the model.
Why defensible	It is a demo.	The corpus and the verifier: an API can be copied in a week, the ten defects you found by measuring cannot.

The four surfaces you will show

#	Surface	What the judge sees	Built
1	Service	curl on a terminal: works, resume bundles, a refusal. JSON, not a screenshot.	Today
2	MCP	Kiro — an AWS product that has never heard of your laptop — asking Evo where it is.	Today, new
3	UI	The same question asked by a human, with the refused works greyed out.	Today
4	Engine	A gate that fails the build when two works fuse.	Months

> **WHAT IS GENUINELY NEW TODAY**
> The repository had no MCP server and no socket listener. Today Evo becomes addressable: an HTTP surface, a protocol any agent can speak, a product UI. "This morning Evo was a library; tonight it is addressable" is true, and true is the point.

The sentence, memorised

Every agent today is told where its work begins and ends. Evo derives it, locally, and lets any agent ask.

If you lose your place, say it again. Everything else is evidence for it.

PART THREE

How it reads Evo

Be able to draw this. The question "how does the new bit read the engine?" is what separates you from someone who wired an API to a mock.

```
sample_events.json      10,392 recorded events
      |
      |   cargo run -p evo-threads --bin boundary_dump
      v
evo-threads ENGINE     Rust, deterministic, offline, no HTTP client
      |
      |   Engine::briefs()  ->  Vec<WorkBrief>
      v
threads.json           the one contract between the old and the new
      |
      |   python3 evo_server.py     (one process, two mouths)
      v
evo_server.py
      |
      +--- serve  :8765   ->  evo_ui.html   (a human asks)
      |
      +--- mcp    stdio   ->  Kiro          (an agent asks)
```

Why each layer exists

Layer	Its job	If you removed it
Engine	Decides the boundary. Deterministic, offline, no model.	There is no product, only a UI.
threads.json	The contract. One file, one shape.	The service would have to link Rust to be useful.
Service	Makes the boundary addressable: HTTP, JSON, a port.	Nothing else can use it. It stays a demo.
MCP	Makes it speakable by agents, not just by your UI.	You are the only client. That is a toy.
Kiro	Proves an outside agent can consume it, on the day.	One less witness; the claim is unchanged.

> **IF A JUDGE ASKS WHERE THE AI IS**
> "Deliberately not in the decision. The boundary has to be deterministic and auditable or it is not a boundary, it is a suggestion. AI consumes this; it does not author it. That is why the answer is the same twice, and why I can show you a test that fails the build when it gets it wrong."

The engine mints two works on this corpus and refuses the other six. That is not a defect: `tests/ground_truth.rs` asserts no fusion, no leisure anchor and determinism — never coverage. Silence is a designed outcome.

PART FOUR

What already exists

Everything below is committed. Your job is to rehearse it, not to rebuild it.

File	What it does	Status
crates/evo-threads/src/bin/boundary_dump.rs	Replays the corpus through the engine, writes threads.json.	Ran: 2 works
exports/ship-happens/evo_server.py	HTTP service and MCP server. Stdlib only.	Tested
exports/ship-happens/evo_ui.html	Product surface, refused rows greyed.	Tested
exports/ship-happens/mcp_probe.py	A real MCP client; backup if Kiro fails.	Tested
exports/ship-happens/boundary.html	Offline explorer, the two break-it buttons.	Tested

.kiro/settings/mcp.json	Registers Evo with Kiro.	Kiro connected

```
architecture-v1/
  .kiro/steering/product.md       the invariant Kiro must not break
  .kiro/settings/mcp.json         registers evo-boundary with Kiro
  crates/evo-threads/src/         the engine — FROZEN today
    bin/boundary_dump.rs          events -> threads.json
  exports/ship-happens/           everything you demo
    evo_server.py                 service + MCP
    evo_ui.html                   product UI
    boundary.html                 offline explorer
    threads.json                  engine output
    boundary.json                 hand-label view
```

> **DO NOT TOUCH THE ENGINE TODAY**
> A rushed edit at 12:40 that does not compile costs the afternoon. If you find a bug, write it down and fix it on Monday.

PART FIVE

Sprint 01: 10:10 to 14:00

Five blocks. Each ends with something you can show, so if you stop at any point you still have a demo. That property matters more than finishing.

Block	Time	You produce	If it fails
A	10:10 – 10:40	Kiro open, engine verified, threads.json written.	Use boundary.json.
B	10:40 – 11:40	Service on :8765, UI loading real works.	Fall back to boundary.html.
C	11:40 – 12:40	Kiro asks Evo and gets evidence back.	Drop MCP; curl the endpoint.
D	12:40 – 13:40	Kiro rehearsal + backup recording.	Drop Render. Never drop the recording.
E	13:40 – 14:00	Frozen, one full run-through, committed.	—

Each block has a twenty-minute overrun limit. Past it, take the failure path, say the fallback line once, and move on. A demo missing its third act still wins. A demo still compiling at 4:10 cannot be submitted.

12:40 – 13:40

Block D — Rehearse the handoff, then insure it

This block changed. Sarvam credits did not land, so there is no voice to build. What replaces it is worth more: the ten seconds where an AWS product asks your engine a question — and the recording that survives if it does not.

D1 · The Kiro rehearsal (25 min)

You are not rehearsing the whole demo. You are rehearsing the handoff, because that is the only part with a dependency you do not control.

```
# terminal 1 — the boundary must be up before Kiro asks
cd ~/Desktop/Evo-Evolution.nosync/exports/ship-happens
python3 evo_server.py serve

# terminal 2 — prove the server is fine, independently of Kiro
python3 mcp_probe.py --tools
```

Then quit Kiro fully (Cmd+Q), reopen it, and run the three demo prompts in order. Time each one. Write the number down — you will use it to pace 0:55.

What to watch	What to do about it
Tool call is instant (<2s)	Good. Give the beat fifteen seconds and move.
Tool call lags 5–10s	Start talking one beat earlier: say "watch it ask" and let it spin — the pause reads as work.
Kiro writes three paragraphs of prose	The prompt failed. Re-paste prompt 2 verbatim; it forbids commentary.
Kiro asks permission every call	Answer once, then re-check autoApprove in .kiro/settings/mcp.json.
MCP panel shows no server	Use mcp_probe.py on stage instead. It is the same protocol, in a terminal.

> **THE FALLBACK IS NOT A DOWNGRADE**
> If Kiro will not connect, run `python3 mcp_probe.py "where did I look for hiking clothes"` and say: "this is the same conversation a moment ago, in a terminal, so you can see the protocol." The judge sees initialize, tools/list, and the answer. It is arguably clearer.

D2 · The backup recording (20 min, non-negotiable)

Your demo now depends on an IDE, a JSON config, and a child process. Record all of it at 13:20, while you are calm and everything works.

```
Cmd-Shift-5  ->  Record Selected Portion  ->  record  ->  Desktop

Cover, in this order:
  1. the service starting, source line says engine+labels
  2. evo_ui.html: the works, the greyed refusals, 2 of 8 claimed
  3. one good question, one refusal
  4. Kiro: list_works, then find_work, then the refusal
  5. boundary.html: merge by inference refused, declare -> exit 1
```

D3 · Render — only if you are ahead (skip by default)

Off the critical path. It cannot touch the boundary honestly: a hosted instance either contradicts local-first or runs fake data. If you do it, it is a synthetic corpus on a public URL — never your real history.

```
python3 evo_server.py serve      # reads $PORT, so it deploys unchanged
# say once: the public one runs synthetic data; the real one runs on this laptop.
```

D4 · Freeze at 13:40

Commit everything. From here you change words, not code. The most common way to lose a hackathon is a change made after the last rehearsal.

PART SIX

Sprint 02: 14:30 to 16:00

Time	Do	Output
14:30 – 15:05	Rehearse the full demo twice, out loud, timed against the stage card.	You finish at 2:45, not 4:10.
15:05 – 15:20	Write the submission. Paste from the next page.	Draft, not a blank form.
15:20 – 15:40	Hostile drill: eight questions, one at a time.	You stop being surprised.
15:40 – 15:52	Rehearse once more; re-record any beat that broke.	Clean recording.
15:52 – 16:00	Submit. Close the lid on code.	Done.

Sprint 02 is where most teams lose, and not because they run out of time. They keep building past the point where building helps. With the internship prize, the marginal rehearsal is worth more than the marginal feature: they are deciding whether they want you on their team.

PASTE INTO CLAUDE / CHATGPT · before 15:40

```
You are a judge at an Indian student hackathon. You are technical, you have
seen forty AI projects today, and you are tired and sceptical. I have three
minutes.

Interrogate me, one question at a time. After each answer say in one line what
a sceptical judge would think of it, then ask the next question. Escalate: what
it does, then accuracy, then why it is not just a folder, then why an
incumbent will not simply build it, then the business.

Do not be encouraging. Stop after eight questions and name my two weakest
answers.
```

PART SEVEN

The submission, field by field

Write it at 15:05, not at 15:58.

Title

```
Evo — the boundary layer for AI agents
```

One line

```
Evo derives where one body of work ends and the next begins, on the device,
so agents can act on a real machine without a human labelling every folder.
```

Description

```
Every AI agent today is TOLD where its work begins and ends: you point it
at a folder. None of them DERIVE it. That is why agents still cannot safely
act on a real desktop - they do not know that this repository is one job and
that deck is another. Scale does not fix it, because the boundary is not in
the content. Two people can have identical files and be doing different jobs.

Evo derives the boundary locally, as a side effect of a product people use
every morning, and exposes it for agents to query.

What you can try in the demo:
- The service: curl a work, its resume bundle, and a refusal.
- The protocol: Kiro, an AWS product, asks Evo where it is over MCP and gets
  an answer with evidence - including the work it nearly picked instead.
- The refusal: six of eight works are refused rather than guessed.
- The gate: declare two works the same and the build fails, on purpose.

Built today: the service, the MCP surface, the product UI.
Built before today: the grouping engine - 16 Rust crates, deterministic, no
HTTP client. We say that plainly because the architecture is the interesting
part, not the timeline.

Track: Kiro (AWS). Sarvam credits did not land, so the integration went to
the tool that composes instead of competing: Kiro consumes the boundary.
```

Stack

```
Rust (16 crates, the engine) · Python 3 standard library (service, MCP,
UI server) · Kiro / AWS (MCP client) · no cloud in the decision path ·
no model decides the boundary
```

> **SAY THE TIMELINE SENTENCE YOURSELF**
> "The engine is months old; today I built the layer that lets you talk to it." One sentence, at the top, in your own voice. It costs nothing and it removes the only way to lose the room.

PART EIGHT

Three minutes, in order

Fullscreen. One tab. Nothing else running. Times are targets, not rules.

	Say	Do
0:00	"Every knowledge worker has a second job nobody pays for: rebuilding what they were doing before the last interruption. It is not a search problem. The answer is not on the web. It is the state of your own machine."	Nothing. Let them look at you.
0:20	"Ten thousand events, one machine, eight separate jobs. I labelled them by hand. The engine independently claimed two of the eight and refused the other six."	Point at the works rail. Greyed rows are refusals.
0:40	"Every agent today is told its boundary — you point it at a folder. None derive it. And content cannot give it to you, because two people can have identical files and different jobs."	curl /works.
0:55	"So I gave the boundary a mouth. Kiro is an AWS product. It has never heard of my laptop. Watch it ask."	Switch to Kiro. Run prompt 1 (list_works).
1:15	"Two of eight. The other six say refused. That is not a bug — a wrong merge is irreversible and a refusal costs nothing."	Scroll the list slowly.
1:30	"Now a real question."	Run prompt 2 (find_work: hiking clothes).
1:45	"Gear, sixty percent. And the last line tells me it nearly picked the trek. A boundary that hides its second guess is not auditable."	Read the rejected line out loud.
2:05	"And when it has no answer—"	Run prompt 3 (capital of France).
2:15	"It refuses."	Say nothing more. Then: "Now break it."
2:20	"—"	boundary.html: merge by inference refused; declare → verifier red, exit 1.
2:40	"That is not a test suite. It is a verifier, and it is the part I would defend."	Hold the red panel.
2:52	"Every agent shipping right now has memory. None of them have jurisdiction. That is the layer."	Stop. Do not fill the silence.

> **THE FOUR WORDS THAT DECIDE IT**
> When the verifier goes red, resist explaining it. Say the sentence, hold the screen, let the room work out what they saw. Explaining steals the realisation, and the realisation is what they repeat to each other.

If you are cut to ninety seconds

Problem in one sentence. Kiro asks once. The refusal. The closing line. Drop the works tour and the break-it. Never drop the refusal.

PART NINE

What they will ask

They ask	You say
"Did you build this today?"	"The engine is months old. Today I built the service, the MCP surface and the UI. I would rather say that than have you find it."

"Why only two works out of eight?"	"Because it would rather be silent than fuse a trek with a jacket. The gate asserts no fusion, no leisure anchor and determinism — it never asserts coverage."
"Why would anyone need this?"	"Their agents are useless on a real machine without it. You cannot hand-label a thousand folders per employee. It has to be derived, once, passively."
"Isn't this just a folder?"	"A folder is declared in advance from a menu. This is derived from behaviour and carries its own evidence. Different artifact, different guarantees."
"Where's the AI?"	"Deliberately not in the decision. A boundary has to be reproducible or it is a suggestion. Models consume it; they do not author it."
"What about Cowork / Recall?"	"They carry context forward. This defines the boundary that context lives inside. A context engine with no jurisdiction is guessing."
"Business model?"	"Per seat, prosumer first. The wedge is people who lose an hour a day to this. I am not going to invent enterprise pricing."
"What would you do with ninety days?"	"Ten people outside the company running it for thirty days. Right now the boundary is proven on exactly one person - me."

> **THE ANSWER THAT MUST NEVER LEAVE YOUR MOUTH**
> "We don't know, here's how we'd find out" is strong. Every invented number is a permanent wound: one judge who knows the domain catches one, and everything else you claimed becomes suspect.

PART TEN

When it breaks, and it will

Symptom	Cause	First minute
Kiro shows no evo-boundary server	Config read at startup only.	Cmd+Q, reopen. Still missing: run mcp_probe.py on stage.
Kiro asks permission on every call	autoApprove not loaded.	Approve once and continue. Do not debug config mid-demo.
Kiro answers in prose, slowly	The prompt was not restrictive enough.	Re-paste prompt 2 verbatim; it forbids commentary.
Service starts but /works is empty	Not run from exports/ship-happens.	cd into it and rerun. It resolves paths from its own file.
Header says 'labels' not 'engine+labels'	threads.json missing or in the wrong folder.	"This is the hand-labelled view" — once — and continue.
Something crashes on stage	Any of the above, live.	Switch to the recording without comment. Keep narrating.
Judge does not understand it	You explained architecture first.	Stop. Problem sentence, then boundary sentence, then the refusal.

The four sentences you must not lose

```
1.  Every agent today is TOLD where its work begins and ends.
2.  None of them DERIVE it, and content cannot give it to them.
3.  Evo derives it locally, and lets any agent ask.
4.  A boundary that guesses is not a boundary.
```

If you can say those four from memory, you cannot lose the room, only the prize.

APPENDIX

Everything in one place

Commands

```
# engine -> json
cargo run -p evo-threads --bin boundary_dump -- \
  --events tools/threads-regression/sample_events.json \
  --out exports/ship-happens/threads.json

# the service + UI
cd exports/ship-happens && python3 evo_server.py serve

# the protocol, without any IDE
python3 evo_server.py mcp
python3 mcp_probe.py "where did I look for hiking clothes"

# the offline explorer (no server, no install)
open boundary.html
```

Endpoints

```
GET  /                the product UI
GET  /works           all works, with claimed/refused flags
GET  /works/{id}      one work with its restore set
GET  /ask?q=...       the boundary answer
GET  /health          source, work count, claimed count
```

MCP tools

```
list_works   every work, marking the ones the engine refused to assert
find_work    which work a resource or phrase belongs to, with evidence
resume_work  the exact surfaces to reopen for a work
```

Numbers you may quote, and only these

```
10,392   recorded events (13-15 September 2026)
   164   resources labelled by hand
     8   bodies of work labelled
     2   claimed by the engine; the other 6 it refuses to assert
     0   HTTP clients in the engine; a test fails the build if one appears
     1   person the boundary is proven on
```

Everything else — users, revenue, retention — is not yours to quote yet. Say "nobody outside the company has run it" and watch how much more they believe the rest.